

Module 8) Theory

Advance Python Programming

1. Printing on Screen

1. Introduction to the print() function in Python.

Definition

The `print()` function is a built-in Python function used to display output on the screen. It helps programmers show messages, variables, calculations, and program results to the user.

Syntax

```
print(object)
```

Examples

Example 1: Printing a message

```
print("Hello, World!")
```

Output:

```
Hello, World!
```

Example 2: Printing a number

```
print(100)
```

Output:

```
100
```

Example 3: Printing multiple values

```
name = "Devendra"  
age = 18
```

```
print("Name:", name)  
print("Age:", age)
```

Output:

```
Name: Devendra  
Age: 18
```

Features of `print()`

1. Displays text, numbers, and variables on the screen.
2. Can print multiple values at the same time.
3. Automatically moves the cursor to the next line after printing.
4. Useful for checking program results and debugging.

Advantages

- Easy to use.
- Helps users see program output.
- Useful for testing and debugging code.

2. Formatting outputs using f-strings and format().

Introduction

In Python, output can be formatted to make it more readable and attractive. Two common methods for formatting output are **f-strings** and the **format()** method. These methods allow us to insert variables and values into strings easily.

1. f-strings (Formatted String Literals)

Definition

An f-string is a string that starts with the letter `f` before the quotation marks. It allows variables and expressions to be inserted directly inside the string using curly braces `{ }`.

Syntax

```
f"Text {variable}"
```

Example

```
name = "Devendra"  
age = 18  
  
print(f"My name is {name} and I am {age} years old.")
```

Output:

My name is Devendra and I am 18 years old.

Advantages of f-strings

1. Easy to read and write.
2. Faster than other formatting methods.
3. Allows expressions inside curly braces.

Example with Expression

```
a = 10  
b = 20  
  
print(f"Sum = {a + b}")
```

Output:

Sum = 30

2. format() Method

Definition

The `format()` method is used to insert values into a string using placeholders `{ }`.

Syntax

```
"Text {}".format(value)
```

Example

```
name = "Devendra"  
age = 18  
  
print("My name is {} and I am {} years old.".format(name, age))
```

Output:

My name is Devendra and I am 18 years old.

Example with Index Numbers

```
print("Name: {0}, Age: {1}".format("Devendra", 18))
```

Output:

Name: Devendra, Age: 18

Advantages of `format()`

1. Easy to insert multiple values.
2. Works in older Python versions.
3. Allows control over the position of values.

Difference Between f-strings and `format()`

f-strings	<code>format()</code>
Introduced in Python 3.6	Available in older versions
Faster and more readable	Slightly longer syntax
Variables written inside <code>{ }</code> directly	Values passed through <code>format()</code>

2. Reading Data from Keyboard

3. Using the `input()` function to read user input from the keyboard.

Introduction

The `input()` function is a built-in Python function used to take input from the user through the keyboard. It allows a program to interact with the user by accepting data during execution.

Definition

The `input()` function reads the data entered by the user and returns it as a string.

Syntax

```
input("Message")
```

- **Message:** A prompt displayed to the user.
 - The value entered by the user is stored in a variable.
-

Example 1: Taking User Input

```
name = input("Enter your name: ")

print("Hello, ", name)
```

Output:

```
Enter your name: Devendra
Hello, Devendra
```

Example 2: Taking Two Inputs

```
city = input("Enter your city: ")
country = input("Enter your country: ")

print("City:", city)
print("Country:", country)
```

Output:

```
Enter your city: Ahmedabad
Enter your country: India

City: Ahmedabad
Country: India
```

Important Note

The `input()` function always returns data as a **string**, even if the user enters a number.

Example

```
age = input("Enter your age: ")

print(age)
print(type(age))
```

Output:

```
Enter your age: 18
18
<class 'str'>
```

Advantages of `input()`

1. Allows interaction between the user and the program.
2. Accepts data during program execution.
3. Easy to use and understand.
4. Useful for creating dynamic programs.

4. Converting user input into different data types (e.g., int, float, etc.).

Introduction

The `input()` function in Python always returns data as a string. To perform mathematical calculations or other operations, we often need to convert the input into different data types such as `int`, `float`, `bool`, or `str`.

Why Type Conversion is Needed

When a user enters a number using `input()`, Python treats it as a string.

Example

```
age = input("Enter your age: ")

print(type(age))
```

Output:

```
<class 'str'>
```

To use the value as a number, we must convert it to the required data type.

1. Converting Input to Integer (`int`)

The `int()` function converts a string into an integer.

Example

```
age = int(input("Enter your age: "))

print("Age:", age)
print(type(age))
```

Output:

```
Enter your age: 18
Age: 18
<class 'int'>
```

Uses of `int()`

- Whole numbers
 - Counting values
 - Mathematical calculations
-

2. Converting Input to Float (`float`)

The `float()` function converts a string into a decimal number.

Example

```
salary = float(input("Enter your salary: "))

print("Salary:", salary)
print(type(salary))
```

Output:

```
Enter your salary: 25000.50
Salary: 25000.5
<class 'float'>
```

Uses of `float()`

- Decimal numbers
 - Measurements
 - Scientific calculations
-

3. Converting Input to String (`str`)

The `str()` function converts a value into a string.

Example

```
num = 100

text = str(num)

print(text)
print(type(text))
```

Output:

```
100
<class 'str'>
```

4. Converting Input to Boolean (`bool`)

The `bool()` function converts a value into either `True` or `False`.

Example

```
num = int(input("Enter a number: "))

print(bool(num))
```

Output 1:

```
Enter a number: 5
True
```

Output 2:

```
Enter a number: 0
False
```

Example: Adding Two Numbers

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

sum = num1 + num2

print("Sum =", sum)
```

Output:

```
Enter first number: 10
Enter second number: 20
Sum = 30
```

Common Type Conversion Functions

Function	Description
<code>int()</code>	Converts value to integer
<code>float()</code>	Converts value to decimal number
<code>str()</code>	Converts value to string
<code>bool()</code>	Converts value to True or False

Advantages of Type Conversion

- 1. Allows mathematical operations on user input.
- 2. Converts data into the required format.
- 3. Makes programs more flexible and useful.
- 4. Helps avoid type-related errors.

3. Opening and Closing Files

5. Opening files in different modes ('r', 'w', 'a', 'r+', 'w+').

Introduction

Python provides the `open()` function to work with files. A file can be opened in different modes depending on whether we want to read, write, append, or perform both operations.

Syntax

```
file = open("filename.txt", "mode")
```

- **filename.txt** → Name of the file.
 - **mode** → Specifies how the file will be used.
-

1. Read Mode ('r')

Definition

The 'r' mode is used to read data from a file. It is the default mode.

Example

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

Features

- Opens a file for reading only.
 - File must already exist.
 - Cannot modify the file.
-

2. Write Mode ('w')

Definition

The 'w' mode is used to write data to a file.

Example

```
file = open("data.txt", "w")
file.write("Hello Python")
file.close()
```

Features

- Creates a new file if it does not exist.
 - If the file exists, old content is deleted.
 - Used for writing new data.
-

3. Append Mode ('a')

Definition

The 'a' mode is used to add data at the end of an existing file.

Example

```
file = open("data.txt", "a")
file.write("\nWelcome to Python")
file.close()
```

Features

- Adds new data without deleting existing content.
 - Creates the file if it does not exist.
 - New data is written at the end of the file.
-

4. Read and Write Mode ('r+')

Definition

The 'r+' mode allows both reading and writing in the same file.

Example

```
file = open("data.txt", "r+")
print(file.read())
file.write("\nNew Data")
file.close()
```

Features

- Supports both reading and writing.
 - File must already exist.
 - Existing content is not deleted.
-

5. Write and Read Mode ('w+')

Definition

The 'w+' mode allows both writing and reading.

Example

```
file = open("data.txt", "w+")
file.write("Python Programming")
file.seek(0)
print(file.read())
file.close()
```

Output

Python Programming

Features

- Supports both writing and reading.
- Creates a new file if it does not exist.
- Deletes existing content before writing.

Comparison of File Modes

Mode	Read	Write	Creates File	Deletes Old Data
'r'	Yes	No	No	No
'w'	No	Yes	Yes	Yes
'a'	No	Yes (Append)	Yes	No
'r+'	Yes	Yes	No	No
'w+'	Yes	Yes	Yes	Yes

Advantages of File Modes

1. Allow different operations on files.
2. Help store and retrieve data permanently.
3. Make file handling flexible and efficient.
4. Support reading, writing, and updating files.

6. Using the open() function to create and access files.

Introduction

The open () function is used in Python to create, open, and access files. It allows a program to read data from a file or write data into a file. File handling is useful for storing information permanently.

Definition

The open () function opens a file and returns a file object that can be used for various file operations.

Syntax

```
file = open("filename.txt", "mode")
```

Where:

- **filename.txt** → Name of the file.
 - **mode** → Specifies how the file will be opened (read, write, append, etc.).
-

Creating a File

A new file can be created using the 'w' mode.

Example

```
file = open("sample.txt", "w")
file.write("Welcome to Python")
file.close()
```

Explanation

- `open()` creates the file if it does not exist.
 - `write()` adds data to the file.
 - `close()` closes the file after use.
-

Accessing a File for Reading

A file can be accessed using the 'r' mode.

Example

```
file = open("sample.txt", "r")
content = file.read()
print(content)
file.close()
```

Output

Welcome to Python

Writing Data to a File

Example

```
file = open("student.txt", "w")
file.write("Name: Devendra")
file.close()
```

This creates a file named **student.txt** and stores the given text in it.

Reading Data from a File

Example

```
file = open("student.txt", "r")
print(file.read())
file.close()
```

Output

Name: Devendra

Appending Data to a File

Example

```
file = open("student.txt", "a")
file.write("\nAge: 18")
file.close()
```

This adds new data at the end of the file without removing the existing content.

Importance of `close()`

After completing file operations, the file should be closed.

```
file.close()
```

Benefits:

- Saves changes properly.
 - Frees system resources.
 - Prevents data loss.
-

Advantages of the `open()` Function

1. Creates new files easily.
2. Allows reading and writing of data.
3. Supports different file modes.
4. Helps store data permanently.
5. Makes file management simple and efficient.

7. Closing files using `close()`.

Introduction

After performing operations on a file, it is important to close the file. In Python, the `close()` method is used to close an opened file. Closing a file releases system resources and ensures that all data is saved properly.

Definition

The `close()` method closes the file that was opened using the `open()` function.

Syntax

```
file.close()
```

Example 1: Closing a File After Reading

```
file = open("data.txt", "r")

content = file.read()
print(content)

file.close()
```

Output

(Displays the contents of the file)

Example 2: Closing a File After Writing

```
file = open("data.txt", "w")

file.write("Hello Python")

file.close()
```

Explanation

- The file is opened in write mode.
 - Data is written into the file.
 - `close()` closes the file and saves the changes.
-

Why is `close()` Important?

1. Saves all changes made to the file.
 2. Releases memory and system resources.
 3. Prevents data loss.
 4. Avoids file corruption.
 5. Improves program performance.
-

Checking Whether a File is Closed

```
file = open("data.txt", "r")

print(file.closed)

file.close()

print(file.closed)
```

Output

```
False
True
```

Explanation

- `False` means the file is open.
 - `True` means the file has been closed.
-

Advantages of Using `close()`

1. Ensures data is stored correctly.
2. Frees system resources.
3. Prevents accidental file modifications.
4. Makes file handling safer and more efficient.

5. Reading and Writing Files

8. Reading from a file using `read()`, `readline()`, `readlines()`.

Introduction

Python provides several methods to read data from a file. The most commonly used methods are `read()`, `readline()`, and `readlines()`. These methods help retrieve file contents in different ways.

1. `read()` Method

Definition

The `read()` method is used to read the entire content of a file at once.

Syntax

```
file.read()
```

Example

```
file = open("data.txt", "r")

content = file.read()
print(content)

file.close()
```

Output

```
Hello Python
Welcome to File Handling
```

Features

- Reads the complete file content.
 - Returns the content as a string.
 - Suitable for small files.
-

2. `readline()` Method

Definition

The `readline()` method reads only one line from the file at a time.

Syntax

```
file.readline()
```

Example

```
file = open("data.txt", "r")

print(file.readline())
print(file.readline())

file.close()
```

Output

Hello Python

Welcome to File Handling

Features

- Reads one line at a time.
- Useful for large files.
- Returns a string containing the line.

3.readlines() Method

Definition

The readlines() method reads all lines from a file and stores them in a list.

Syntax

```
file.readlines()
```

Example

```
file = open("data.txt", "r")

lines = file.readlines()
print(lines)

file.close()
```

Output

['Hello Python\n', 'Welcome to File Handling\n']

Features

- Reads all lines at once.
- Returns a list of strings.
- Each line becomes an element of the list.

Comparison of Reading Methods

Method	Description	Return Type
read()	Reads the entire file	String
readline()	Reads one line at a time	String
readlines()	Reads all lines and stores them in a list	List

Advantages

1. Easy to read data from files.
2. Different methods suit different needs.

- Helps process text files efficiently.
- Useful for data storage and retrieval.

9. Writing to a file using write() and writelines().

Introduction

Python provides the `write()` and `writelines()` methods to store data in a file. These methods are used when we want to save text permanently in a file.

1. write() Method

Definition

The `write()` method is used to write a string to a file.

Syntax

```
file.write(data)
```

- `data` is the text that will be written to the file.

Example

```
file = open("data.txt", "w")
```

```
file.write("Hello Python")
```

```
file.close()
```

Output in File

```
Hello Python
```

Writing Multiple Lines

```
file = open("data.txt", "w")
```

```
file.write("Hello Python\n")
```

```
file.write("Welcome to File Handling")
```

```
file.close()
```

Output in File

```
Hello Python
```

```
Welcome to File Handling
```

Features of write()

- Writes one string at a time.
- Returns the number of characters written.
- Useful for writing small amounts of data.

2. writelines() Method

Definition

The `writelines()` method is used to write multiple strings to a file at once.

Syntax

```
file.writelines(list_of_strings)
```

Example

```
file = open("data.txt", "w")

lines = ["Python\n", "Java\n", "C++\n"]

file.writelines(lines)

file.close()
```

Output in File

Python
Java
C++

Features of writelines ()

- 1. Writes multiple lines at once.
- 2. Takes a list of strings as input.
- 3. Does not automatically add newline characters (\n).

Difference Between write () and writelines ()

write ()	writelines ()
Writes a single string	Writes multiple strings
Used for one line or one text block	Used for a list of lines
Takes a string as argument	Takes a list of strings as argument

Advantages

- 1. Helps store data permanently in files.
- 2. Makes file handling easy and efficient.
- 3. Supports writing single or multiple lines.
- 4. Useful for saving program output and records.

5.Exception Handling

10.Introduction to exceptions and how to handle them using try, except, and finally.

Introduction

An **exception** is an error that occurs during the execution of a program. If an exception is not handled, the program stops running. Python provides exception handling using try, except, and finally blocks to prevent the program from crashing.

What is an Exception?

An exception is an unexpected event or error that occurs while a program is running.

Example

```
num = 10
print(num / 0)
```

Output

```
ZeroDivisionError: division by zero
```

The program stops because division by zero is not allowed.

1. The `try` Block

Definition

The `try` block contains code that may cause an exception.

Syntax

```
try:  
    # code that may cause an exception
```

Example

```
try:  
    num = 10 / 0  
    print(num)  
except:  
    print("An error occurred")
```

Output

```
An error occurred
```

2. The `except` Block

Definition

The `except` block handles the exception if an error occurs in the `try` block.

Syntax

```
try:  
    # risky code  
except:  
    # error handling code
```

Example

```
try:  
    number = int(input("Enter a number: "))  
    print(number)  
except:  
    print("Invalid input")
```

Output

```
Enter a number: abc  
Invalid input
```

3. The `finally` Block

Definition

The `finally` block is always executed whether an exception occurs or not.

Syntax

```
try:
    # code
except:
    # exception handling
finally:
    # always executed
```

Example

```
try:
    num = 10 / 2
    print(num)
except:
    print("Error occurred")
finally:
    print("Program ended")
```

Output

```
5.0
Program ended
```

Complete Example

```
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))

    result = num1 / num2
    print("Result =", result)

except ZeroDivisionError:
    print("Cannot divide by zero")

except ValueError:
    print("Please enter valid numbers")

finally:
    print("Execution completed")
```

Possible Output 1

```
Enter first number: 10
Enter second number: 0
Cannot divide by zero
Execution completed
```

Possible Output 2

```
Enter first number: abc
Please enter valid numbers
Execution completed
```

Advantages of Exception Handling

1. Prevents the program from crashing.
2. Handles errors gracefully.
3. Improves program reliability.
4. Makes debugging easier.
5. Ensures important code runs using `finally`.

11.Understanding multiple exceptions and custom exceptions.

Introduction

In Python, a program may face different types of errors during execution. We can handle multiple exceptions using multiple `except` blocks. Python also allows programmers to create their own exceptions, called **custom exceptions**, for specific situations.

1. Multiple Exceptions

Definition

Multiple exceptions are used when a program may generate different types of errors. Each error can be handled using a separate `except` block.

Syntax

```
try:
    # code
except ExceptionType1:
    # handle first exception
except ExceptionType2:
    # handle second exception
```

Example of Multiple Exceptions

```
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))

    result = num1 / num2
    print("Result =", result)

except ZeroDivisionError:
    print("Cannot divide by zero")

except ValueError:
    print("Please enter valid numbers")
```

Output 1

```
Enter first number: 10
Enter second number: 0
Cannot divide by zero
```

Output 2

```
Enter first number: abc
Please enter valid numbers
```

Advantages of Multiple Exceptions

1. Handles different errors separately.
 2. Makes programs more reliable.
 3. Provides specific error messages.
 4. Improves debugging and maintenance.
-

2. Custom Exceptions

Definition

A custom exception is a user-defined exception created by the programmer for specific conditions in a program.

Custom exceptions are created by inheriting from the built-in `Exception` class.

Syntax

```
class MyException(Exception):  
    pass
```

Example of Custom Exception

```
class AgeError(Exception):  
    pass  
  
age = int(input("Enter age: "))  
  
try:  
    if age < 18:  
        raise AgeError("Age must be 18 or above")  
    print("You are eligible")  
  
except AgeError as e:  
    print(e)
```

Output 1

```
Enter age: 15  
Age must be 18 or above
```

Output 2

```
Enter age: 20  
You are eligible
```

Using `raise` Keyword

Definition

The `raise` keyword is used to generate an exception manually.

Example

```
age = 15

if age < 18:
    raise Exception("Not eligible for voting")
```

Output

Exception: Not eligible for voting

Difference Between Multiple Exceptions and Custom Exceptions

Multiple Exceptions	Custom Exceptions
Handles built-in errors	Creates user-defined errors
Uses multiple <code>except</code> blocks	Uses a class derived from <code>Exception</code>
Examples: <code>ValueError</code> , <code>ZeroDivisionError</code>	Example: <code>AgeError</code> , <code>SalaryError</code>

Advantages of Custom Exceptions

- 1. Creates meaningful error messages.
- 2. Handles application-specific problems.
- 3. Makes programs easier to understand.
- 4. Improves code readability and maintenance.

5. Class and Object (OOP Concepts)

12.Understanding the concepts of classes, objects, attributes, and methods in Python.

Introduction

Python is an Object-Oriented Programming (OOP) language. OOP helps organize code using classes and objects. The main concepts of OOP are **classes, objects, attributes, and methods**.

1. Class

Definition

A class is a blueprint or template used to create objects. It defines the properties (attributes) and behaviors (methods) that objects will have.

Syntax

```
class ClassName:
    pass
```

Example

```
class Student:
    pass
```

Here, `Student` is a class.

2. Object

Definition

An object is an instance of a class. It is created from a class and can access its attributes and methods.

Example

```
class Student:
    pass
```

```
s1 = Student()
print(s1)
```

Here, s1 is an object of the Student class.

3. Attributes

Definition

Attributes are variables that belong to a class or object. They store data about the object.

Example

```
class Student:
    name = "Devendra"
    age = 18
```

```
s1 = Student()
```

```
print(s1.name)
print(s1.age)
```

Output

```
Devendra
18
```

Here, name and age are attributes.

4. Methods

Definition

Methods are functions defined inside a class. They describe the behavior of objects.

Example

```
class Student:
    def display(self):
        print("Welcome to Python")
```

```
s1 = Student()
s1.display()
```

Output

```
Welcome to Python
```

Here, `display()` is a method.

Complete Example

```
class Student:

    name = "Devendra"
    age = 18

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)

s1 = Student()
s1.display()
```

Output

Name: Devendra
Age: 18

Explanation of `self`

- `self` refers to the current object.
- It is used to access attributes and methods of the class.
- It must be the first parameter of every instance method.

Example

```
class Student:

    def show(self):
        print("This is self example")

s1 = Student()
s1.show()
```

Relationship Between Class, Object, Attribute, and Method

Concept	Description
Class	Blueprint for creating objects
Object	Instance of a class
Attribute	Variable that stores data
Method	Function that defines behavior

Advantages of OOP

1. Makes code organized and reusable.
2. Improves readability.
3. Helps manage large programs.
4. Supports real-world modeling.

5. Reduces code duplication.

13.Difference between local and global variables.

Introduction

Variables in Python can be classified as **local variables** and **global variables** based on where they are declared. Understanding the difference between them is important for managing data within a program.

1. Local Variable

Definition

A local variable is a variable declared inside a function. It can only be accessed within that function.

Example

```
def display():  
    name = "Devendra"    # Local variable  
    print(name)
```

```
display()
```

Output

Devendra

Features of Local Variables

1. Declared inside a function.
 2. Accessible only within that function.
 3. Created when the function is called.
 4. Destroyed when the function ends.
-

2. Global Variable

Definition

A global variable is a variable declared outside all functions. It can be accessed from anywhere in the program.

Example

```
name = "Devendra"    # Global variable
```

```
def display():  
    print(name)
```

```
display()
```

```
print(name)
```

Output

Devendra
Devendra

Features of Global Variables

1. Declared outside functions.
2. Accessible throughout the program.
3. Exists until the program ends.
4. Can be used by multiple functions.

Using the global Keyword

If we want to modify a global variable inside a function, we use the `global` keyword.

Example

```
count = 10

def update():
    global count
    count = 20

update()

print(count)
```

Output

20

Difference Between Local and Global Variables

Local Variable	Global Variable
Declared inside a function	Declared outside a function
Accessible only within that function	Accessible throughout the program
Exists only during function execution	Exists until the program ends
Cannot be used outside the function	Can be used inside and outside functions

Example Showing Both

```
name = "Python"    # Global variable

def show():
    course = "Programming"    # Local variable
    print("Global Variable:", name)
    print("Local Variable:", course)

show()

print("Global Variable:", name)
```

Output

Global Variable: Python
Local Variable: Programming
Global Variable: Python

Advantages

Local Variables

- 1. Reduce memory usage.
- 2. Prevent accidental changes from other functions.
- 3. Improve program security.

Global Variables

- 1. Can be shared by multiple functions.
- 2. Reduce the need to pass variables repeatedly.

- 3. Useful for storing common data.

7. Inheritance

14. Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python.

Introduction

Inheritance is an important feature of Object-Oriented Programming (OOP). It allows one class to inherit the properties and methods of another class. This helps in code reusability and reduces duplication.

1. Single Inheritance

Definition

In Single Inheritance, one child class inherits from one parent class.

Example

```
class Parent:
    def show(self):
        print("This is Parent class")

class Child(Parent):
    pass

obj = Child()
obj.show()
```

Output

This is Parent class

Diagram

```
Parent
 |
Child
```

2. Multilevel Inheritance

Definition

In Multilevel Inheritance, a class inherits from another class, and then another class inherits from it.

Example

```
class Grandparent:
    def display1(self):
        print("Grandparent Class")

class Parent(Grandparent):
    def display2(self):
        print("Parent Class")
```

```
class Child(Parent):
    def display3(self):
        print("Child Class")
```

```
obj = Child()
```

```
obj.display1()
obj.display2()
obj.display3()
```

Output

```
Grandparent Class
Parent Class
Child Class
```

Diagram

```
Grandparent
  |
  Parent
  |
  Child
```

3. Multiple Inheritance

Definition

In Multiple Inheritance, a child class inherits from more than one parent class.

Example

```
class Father:
    def father_method(self):
        print("Father Class")
```

```
class Mother:
    def mother_method(self):
        print("Mother Class")
```

```
class Child(Father, Mother):
    pass
```

```
obj = Child()
```

```
obj.father_method()
obj.mother_method()
```

Output

```
Father Class
Mother Class
```

Diagram

```
Father    Mother
  \        /
   \      /
    Child
```

4. Hierarchical Inheritance

Definition

In Hierarchical Inheritance, multiple child classes inherit from a single parent class.

Example

```
class Parent:
    def show(self):
        print("Parent Class")

class Child1(Parent):
    pass

class Child2(Parent):
    pass

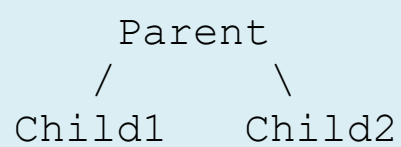
obj1 = Child1()
obj2 = Child2()

obj1.show()
obj2.show()
```

Output

```
Parent Class
Parent Class
```

Diagram



5. Hybrid Inheritance

Definition

Hybrid Inheritance is a combination of two or more types of inheritance such as multiple and multilevel inheritance.

Example

```
class A:
    def methodA(self):
        print("Class A")

class B(A):
    def methodB(self):
        print("Class B")

class C(A):
    def methodC(self):
        print("Class C")
```

```
class D(B, C):
    def methodD(self):
        print("Class D")

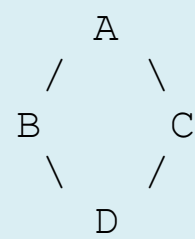
obj = D()

obj.methodA()
obj.methodB()
obj.methodC()
obj.methodD()
```

Output

Class A
Class B
Class C
Class D

Diagram



Comparison of Inheritance Types

Type	Description
Single Inheritance	One child inherits from one parent
Multilevel Inheritance	Inheritance in multiple levels
Multiple Inheritance	One child inherits from multiple parents
Hierarchical Inheritance	Multiple children inherit from one parent
Hybrid Inheritance	Combination of two or more inheritance types

Advantages of Inheritance

- 1. Promotes code reusability.
- 2. Reduces code duplication.
- 3. Makes programs easier to maintain.
- 4. Supports hierarchical classification.
- 5. Improves program organization.

15.Using the super() function to access properties of the parent class.

Introduction

In Python, the `super()` function is used to access the methods and properties of the parent class from the child class. It helps in inheritance by allowing the child class to reuse the code of the parent class.

What is super () ?

Definition

The `super ()` function returns a temporary object of the parent class, which allows the child class to call the parent class methods and constructors.

Syntax

```
super().method_name()
```

Example 1: Accessing a Parent Class Method

```
class Parent:
    def show(self):
        print("This is Parent Class")

class Child(Parent):
    def display(self):
        super().show()
        print("This is Child Class")

obj = Child()

obj.display()
```

Output

```
This is Parent Class
This is Child Class
```

Explanation

- `super().show()` calls the `show()` method of the parent class.
 - The child class can use the parent class method without rewriting it.
-

Example 2: Using super () with Constructors

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name)
        self.age = age

obj = Child("Devendra", 18)

print("Name:", obj.name)
print("Age:", obj.age)
```

Output

```
Name: Devendra
Age: 18
```

Explanation

- `super().__init__(name)` calls the parent class constructor.
- The child class gets the `name` attribute from the parent class.

Example 3: Accessing Parent and Child Methods

```
class Person:
    def show(self):
        print("Person Details")

class Student(Person):
    def show_student(self):
        super().show()
        print("Student Details")

obj = Student()
obj.show_student()
```

Output

```
Person Details
Student Details
```

Advantages of `super()`

1. Reuses parent class code.
 2. Reduces code duplication.
 3. Makes inheritance easier to manage.
 4. Improves code readability.
 5. Helps access parent class constructors and methods.
-

Difference Between Using Parent Class Name and `super()`

Parent Class Name	<code>super()</code>
Directly calls the parent class	Automatically refers to the parent class
Less flexible	More flexible and recommended
Requires parent class name	No need to write parent class name

8. Method Overloading and Overriding

16. Method overloading: defining multiple methods with the same name but different parameters.

Introduction

Method overloading is a feature where multiple methods have the same name but different numbers or types of parameters. Python does not support method overloading directly like some other programming languages, but it can be achieved using default arguments or variable-length arguments.

What is Method Overloading?

Definition

Method overloading means creating multiple methods with the same name that perform different tasks based on the parameters passed to them.

Method Overloading Using Default Arguments

Example

```
class Addition:

    def add(self, a, b=0, c=0):
        print("Sum =", a + b + c)

obj = Addition()

obj.add(10)
obj.add(10, 20)
obj.add(10, 20, 30)
```

Output

```
Sum = 10
Sum = 30
Sum = 60
```

Explanation

- When one argument is passed, only `a` is used.
 - When two arguments are passed, `a` and `b` are used.
 - When three arguments are passed, all values are added.
-

Method Overloading Using Variable-Length Arguments

Example

```
class Addition:

    def add(self, *numbers):
        print("Sum =", sum(numbers))

obj = Addition()

obj.add(10)
obj.add(10, 20)
obj.add(10, 20, 30, 40)
```

Output

```
Sum = 10
Sum = 30
Sum = 100
```

Explanation

- `*numbers` accepts any number of arguments.
 - The `sum()` function adds all the values.
-

Why Python Does Not Support Traditional Method Overloading

Consider the following code:

```
class Test:

    def show(self):
        print("First Method")

    def show(self, name):
        print("Hello", name)

obj = Test()
obj.show("Devendra")
```

Output

Hello Devendra

Explanation

- The second show () method replaces the first one.
- Python keeps only the latest method definition with the same name.

Advantages of Method Overloading

1. Improves code readability.
2. Allows a method to handle different inputs.
3. Reduces the need for multiple method names.
4. Makes programs more flexible.

Difference Between Method Overloading and Method Overriding

Method Overloading	Method Overriding
Same method name with different parameters	Same method name in parent and child classes
Usually in the same class	Occurs in inheritance
Achieved using default or variable arguments in Python	Directly supported in Python

17. Method overriding: redefining a parent class method in the child class.

Introduction

Method overriding is an important concept in Object-Oriented Programming (OOP). It occurs when a child class provides its own implementation of a method that is already defined in the parent class. This allows the child class to modify or extend the behavior of the parent class method.

What is Method Overriding?

Definition

Method overriding is the process of redefining a method in the child class that already exists in the parent class with the same name and parameters.

Syntax

```
class Parent:
    def show(self):
        print("Parent Method")

class Child(Parent):
    def show(self):
        print("Child Method")
```

Example 1: Basic Method Overriding

```
class Parent:
    def show(self):
        print("This is Parent Class")

class Child(Parent):
    def show(self):
        print("This is Child Class")

obj = Child()
obj.show()
```

Output

This is Child Class

Explanation

- Both classes have a method named `show()`.
 - The child class method overrides the parent class method.
 - When `show()` is called using the child object, the child version executes.
-

Example 2: Parent and Child Methods

```
class Animal:
    def sound(self):
        print("Animals make sounds")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

obj = Dog()
obj.sound()
```

Output

Dog barks

Accessing Parent Method Using `super()`

```
class Parent:
    def show(self):
        print("Parent Method")

class Child(Parent):
    def show(self):
        super().show()
```

```
        print("Child Method")

obj = Child()
obj.show()
```

Output

```
Parent Method
Child Method
```

Explanation

- `super().show()` calls the parent class method.
- Then the child class method executes.

Difference Between Method Overloading and Method Overriding

Method Overloading	Method Overriding
Same method name with different parameters	Same method name and parameters
Usually within the same class	Between parent and child classes
Not directly supported in Python	Fully supported in Python
Increases flexibility	Changes inherited behavior

Advantages of Method Overriding

1. Allows customization of inherited methods.
2. Supports code reusability.
3. Improves flexibility in programs.
4. Helps implement polymorphism.
5. Makes code easier to maintain.

10. SQLite3 and PyMySQL (Database Connectors)

18. Introduction to SQLite3 and PyMySQL for database connectivity.

Introduction

Databases are used to store and manage data efficiently. Python provides various modules to connect with databases. Two commonly used modules are **SQLite3** and **PyMySQL**. These modules allow Python programs to create, access, and manage databases.

1. SQLite3

Definition

SQLite3 is a built-in Python module used to work with SQLite databases. It does not require a separate database server and stores data in a single file.

Features of SQLite3

1. Built into Python.
2. No installation required.

3. Lightweight and fast.
4. Stores data in a single file.
5. Suitable for small and medium-sized applications.

Connecting to SQLite Database

```
import sqlite3

conn = sqlite3.connect("student.db")

print("Database Connected")

conn.close()
```

Output

Database Connected

Explanation

- `import sqlite3` imports the SQLite module.
- `connect()` creates or opens a database.
- `close()` closes the connection.

Creating a Table in SQLite3

```
import sqlite3

conn = sqlite3.connect("student.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE Student(
    id INTEGER PRIMARY KEY,
    name TEXT
)
""")

conn.commit()
conn.close()
```

2. PyMySQL

Definition

PyMySQL is a Python library used to connect Python programs with MySQL databases.

Features of PyMySQL

1. Connects Python with MySQL.
2. Supports SQL queries.
3. Useful for large applications.
4. Provides secure database access.
5. Works with MySQL servers.

Installing PyMySQL

```
pip install pymysql
```

Connecting to MySQL Database

```
import pymysql

conn = pymysql.connect(
    host="localhost",
    user="root",
    password="1234",
    database="studentdb"
)

print("Database Connected")

conn.close()
```

Output

Database Connected

Explanation

- `host` specifies the server location.
- `user` specifies the MySQL username.
- `password` specifies the MySQL password.
- `database` specifies the database name.

Difference Between SQLite3 and PyMySQL

SQLite3	PyMySQL
Built into Python	Requires installation
Uses SQLite database	Uses MySQL database
No server required	Requires MySQL server
Suitable for small projects	Suitable for large projects
Data stored in a file	Data stored on a database server

Advantages of Database Connectivity

1. Stores data permanently.
2. Makes data management easier.
3. Supports data retrieval and updates.
4. Improves application performance.
5. Allows handling large amounts of data.

19.Creating and executing SQL queries from Python using these connectors.

Introduction

SQL (Structured Query Language) is used to interact with databases. Python can execute SQL queries using database connectors such as **SQLite3** and **PyMySQL**. These connectors allow Python programs to create tables, insert records, retrieve data, update data, and delete records.

1. Executing SQL Queries Using SQLite3

Creating a Table

```
import sqlite3

conn = sqlite3.connect("student.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE Student(
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER
)
""")

conn.commit()
conn.close()
```

Explanation

- `cursor.execute()` executes the SQL query.
 - `commit()` saves the changes.
 - `close()` closes the database connection.
-

Inserting Data

```
import sqlite3

conn = sqlite3.connect("student.db")

cursor = conn.cursor()

cursor.execute(
    "INSERT INTO Student(name, age) VALUES('Devendra', 18)"
)

conn.commit()
conn.close()
```

Retrieving Data

```
import sqlite3

conn = sqlite3.connect("student.db")

cursor = conn.cursor()

cursor.execute("SELECT * FROM Student")

rows = cursor.fetchall()

for row in rows:
    print(row)
```

```
conn.close()
```

Output

```
(1, 'Devendra', 18)
```

2. Executing SQL Queries Using PyMySQL

Creating a Table

```
import pymysql

conn = pymysql.connect(
    host="localhost",
    user="root",
    password="1234",
    database="studentdb"
)

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE Student(
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    age INT
)
""")

conn.commit()
conn.close()
```

Inserting Data

```
import pymysql

conn = pymysql.connect(
    host="localhost",
    user="root",
    password="1234",
    database="studentdb"
)

cursor = conn.cursor()

sql = "INSERT INTO Student(name, age) VALUES(%s, %s)"
values = ("Devendra", 18)

cursor.execute(sql, values)

conn.commit()
conn.close()
```

Retrieving Data

```
import pymysql

conn = pymysql.connect(
    host="localhost",
    user="root",
    password="1234",
    database="studentdb"
)

cursor = conn.cursor()

cursor.execute("SELECT * FROM Student")

rows = cursor.fetchall()

for row in rows:
    print(row)

conn.close()
```

Output

```
(1, 'Devendra', 18)
```

Common SQL Queries

SQL Query	Purpose
CREATE TABLE	Creates a table
INSERT INTO	Inserts records
SELECT	Retrieves data
UPDATE	Updates records
DELETE	Deletes records

Example of UPDATE Query

```
cursor.execute(
    "UPDATE Student SET age = 19 WHERE name = 'Devendra'"
)
```

Example of DELETE Query

```
cursor.execute(
    "DELETE FROM Student WHERE name = 'Devendra'"
)
```

Advantages

1. Enables Python to communicate with databases.
2. Supports data storage and retrieval.
3. Allows efficient data management.

- 4. Helps build database-driven applications.
- 5. Supports SQL operations directly from Python.

11. Search and Match Functions

20. Using re.search() and re.match() functions in Python's re module for pattern matching.

Introduction

The **re (Regular Expression)** module in Python is used for pattern matching in strings. It helps search, match, and manipulate text efficiently. Two commonly used functions are **re.search()** and **re.match()**.

1. re.search()

Definition

The `re.search()` function searches the entire string for a specified pattern. If the pattern is found, it returns a match object; otherwise, it returns `None`.

Syntax

```
re.search(pattern, string)
```

Example 1: Using re.search()

```
import re

text = "Welcome to Python Programming"

result = re.search("Python", text)

if result:
    print("Pattern Found")
else:
    print("Pattern Not Found")
```

Output

Pattern Found

Example 2: User Input with re.search()

```
import re

text = input("Enter a sentence: ")
word = input("Enter a word to search: ")

result = re.search(word, text)

if result:
    print("Word Found")
```

```
else:  
    print("Word Not Found")
```

Sample Output

```
Enter a sentence: I am learning Python  
Enter a word to search: Python  
Word Found
```

Features of `re.search()`

1. Searches the entire string.
 2. Returns the first match found.
 3. Returns `None` if no match exists.
 4. Useful for finding patterns anywhere in the text.
-

2. `re.match()`

Definition

The `re.match()` function checks whether the pattern matches at the beginning of the string.

Syntax

```
re.match(pattern, string)
```

Example 1: Using `re.match()`

```
import re  
  
text = "Python Programming"  
  
result = re.match("Python", text)  
  
if result:  
    print("Match Found")  
else:  
    print("Match Not Found")
```

Output

```
Match Found
```

Example 2: Pattern Not at the Beginning

```
import re  
  
text = "I love Python"  
  
result = re.match("Python", text)  
  
if result:  
    print("Match Found")
```

```
else:
    print("Match Not Found")
```

Output

Match Not Found

Example 3: User Input with `re.match()`

```
import re

text = input("Enter a sentence: ")
word = input("Enter a word to match: ")

result = re.match(word, text)

if result:
    print("Match Found at Beginning")
else:
    print("Match Not Found at Beginning")
```

Difference Between `re.search()` and `re.match()`

<code>re.search()</code>	<code>re.match()</code>
Searches the entire string	Searches only at the beginning
Finds pattern anywhere in text	Finds pattern only at the start
More flexible	More restrictive
Returns first match found	Returns match only if start matches

Example Comparing Both

```
import re

text = "I am learning Python"

print(re.search("Python", text))
print(re.match("Python", text))
```

Output

<re.Match object>
None

Explanation

- `re.search()` finds **Python** anywhere in the string.
 - `re.match()` fails because **Python** is not at the beginning.
-

Advantages of Regular Expressions

- Makes pattern searching easy.
- Saves time when processing text.

3. Useful for data validation.
4. Helps search large amounts of text.
5. Improves text-processing efficiency.

21. Difference between search and match.

Introduction

The `re.search()` and `re.match()` functions are used in Python's **re (Regular Expression)** module for pattern matching. Although both are used to find patterns in strings, they work differently.

`re.search()`

Definition

`re.search()` searches the **entire string** for a pattern. It returns the first match found anywhere in the string.

Example

```
import re

text = "I am learning Python"

result = re.search("Python", text)

if result:
    print("Pattern Found")
```

Output

Pattern Found

`re.match()`

Definition

`re.match()` checks for a pattern **only at the beginning** of the string.

Example

```
import re

text = "I am learning Python"

result = re.match("Python", text)

if result:
    print("Match Found")
else:
    print("Match Not Found")
```

Output

Match Not Found

Difference Between re.search() and re.match()

re.search()	re.match()
Searches the entire string	Searches only at the beginning of the string
Finds a pattern anywhere in the text	Finds a pattern only at the start
More flexible	More strict
Returns first matching object	Returns match object only if start matches
Commonly used for general searching	Commonly used for checking starting patterns

Example Comparing Both

```
import re

text = "Welcome to Python"

print(re.search("Python", text))
print(re.match("Python", text))
```

Output

<re.Match object>
None

Explanation

- re.search("Python", text) finds **Python** anywhere in the string.
- re.match("Python", text) returns None because the string does not start with **Python**.

Another Example

```
import re

text = "Python Programming"

print(re.search("Python", text))
print(re.match("Python", text))
```

Output

<re.Match object>
<re.Match object>

Explanation

Since **Python** is at the beginning of the string, both functions return a match.

Advantages

- Useful for pattern matching.
- Helps validate user input.
- Makes text searching easier.
- Saves time when processing strings.

